

CSR Sparse-Dense Matrix Multiplication on CPUs with RHS-Width Robustness and a Loopline-Guided Performance Model

Autonomous Research Infrastructure

June 12, 2026

Abstract

Sparse-dense matrix multiplication (SpMM) in CSR format is a core kernel in scientific computing and modern sparse learning pipelines, yet its performance on CPUs is highly sensitive to the width k of the dense right-hand side (RHS) matrix. We present a CSR SpMM implementation with a cache-aware kernel structure and a store-policy selector intended to switch between regular temporal stores and non-temporal stores as k varies. We validate correctness against an independent dense FP32 reference, achieving a maximum absolute error of 6.78958×10^{-6} .

1 Introduction

Sparse-dense matrix multiplication (SpMM), $C \leftarrow AB$ with sparse A and dense B , is widely used in simulation codes, signal processing, and learning workloads. In CSR SpMM, the irregular access pattern induced by the sparse structure interacts with cache hierarchies and store buffers; a particularly important practical issue is that performance can change drastically when the RHS width k changes (e.g., small k behaves like SpMV, while larger k exposes more reuse in B). This sensitivity complicates performance portability and makes it difficult to choose optimizations such as non-temporal stores and blocking strategies.

This paper targets a CPU implementation of CSR SpMM with a cache-aware kernel structure and a store-policy selector intended to adapt to varying k , and it outlines a measurement methodology intended to support future performance modeling (e.g., identifying bandwidth- vs. compute-limited regimes). Our work is motivated by the broader importance of sparse matrix kernels Dorrance et al. [2014], Alappat et al. [2020] and by recent interest in high-performance SpMM on modern Arm CPUs Lei et al. [2025].

Contributions.

- We implement a CSR SpMM kernel and a store-policy selector intended to choose between regular temporal stores and non-temporal stores as a function of k (Figure 1 illustrates the design goal).
- We provide a reproducible measurement methodology that includes correctness validation against an independent dense FP32 reference, plus optional microbenchmarks (bandwidth probes and a compute-inflation probe) intended to support future performance modeling.
- We provide an experimental setup for collecting artifacts on aarch64 and x86_64 to support future characterization and model validation Alappat et al. [2020], Dufek et al. [2021].

2 Related Work

Sparse matrix kernels have a long history of architecture-aware optimization, where performance is often limited by memory bandwidth and irregular access rather than peak FLOPs. Foundational studies of sparse kernels emphasize scalable implementations and the role of memory systems Dorrance et al. [2014]. On recent aarch64 HPC systems, performance modeling for streaming kernels and sparse operations highlights the importance of measuring attainable bandwidth ceilings and accounting for architectural details beyond nominal peak values Alappat et al. [2020]. Broader analyses of sparse kernel behavior across methods and datasets also reinforce that irregular access patterns can dominate runtime and complicate prediction Dhandhanian et al. [2021].

For sparse matrix-matrix multiplication, modern work explores format choices and heterogeneity-aware strategies Cheng et al. [2023], Namjoo et al. [2026]. While much of this literature focuses on SpGEMM or GPU/heterogeneous settings, the underlying challenge of selecting appropriate kernels and data layouts remains central on CPUs, particularly as k changes. Recent work on Arm architectures argues that careful microarchitectural tuning is required for high SpMM performance Lei et al. [2025]. Our work complements these efforts by focusing on implementing mechanisms intended to improve k -robustness in CSR SpMM (notably a store-policy selector), and by providing a lightweight measurement methodology intended to support future model development and validation.

3 Methodology

We compute $C \in \mathbb{R}^{M \times k}$ from a CSR matrix $A \in \mathbb{R}^{M \times K}$ and a dense RHS $B \in \mathbb{R}^{K \times k}$:

$$C_{i,:} \leftarrow \sum_{p=\text{rowptr}[i]}^{\text{rowptr}[i+1]-1} \text{val}[p] \cdot B_{\text{col}[p],:}. \quad (1)$$

The implementation is parallelized over CSR rows using OpenMP with a fixed thread count.

3.1 CSR SpMM kernel

The baseline kernel follows the standard CSR SpMM structure: for each row i , iterate over the nonzeros, gather the corresponding RHS row of B , and perform a scaled vector add into $C_{i,:}$. The critical design choice for k -robustness is how the kernel initializes and stores C , because C is dense and potentially large; depending on k , write-allocate and cache pollution can dominate.

We consider two store policies:

- **Regular temporal stores:** standard writes to C , benefiting from cache when reuse exists.
- **Non-temporal stores:** streaming stores to reduce cache pollution and avoid write-allocate traffic for large k .

A selector chooses the policy based on an empirical tuning procedure over k (Section 3.4); in this revision we describe the mechanism, but do not claim artifact-grounded crossover values.

3.2 Correctness validation

Correctness is validated by comparing the CSR SpMM output against an independent dense reference implementation (dense A constructed from CSR) in FP32. FP64 validation is an optional

check when enabled, but FP64 error results are not reported as artifact-grounded outcomes in this revision. The reported metric is the maximum absolute elementwise error:

$$\epsilon_{\max} = \max_{i,j} \left| C_{ij}^{(\text{spmm})} - C_{ij}^{(\text{ref})} \right|.$$

3.3 Microbenchmarks and ceilings

To parameterize the performance model, we measure:

- **DRAM memcpy bandwidth** and related in-code memory ceilings (used as bandwidth ceilings).
- **STREAM triad bandwidth** as an additional indicator of sustainable bandwidth.
- **Compute inflation probe**: an artificial increase in arithmetic intensity by adding a fixed number of fused multiply-add operations (FMAs) per element of B . This yields an estimate of an effective compute rate f_{eff} (FLOP/s) that can be used as a compute ceiling.

This style of combining attainable bandwidth measurements with compute ceilings follows the spirit of roofline/loopline modeling Alappat et al. [2020], Dufek et al. [2021].

3.4 Store-policy selector

For a fixed sparse matrix A and varying k , we benchmark both store policies and identify an empirical crossover k at which non-temporal stores become faster than regular stores. The selector then chooses:

$$\text{policy}(k) = \begin{cases} \text{regular} & k < k^* \\ \text{non-temporal} & k \geq k^* \end{cases}$$

Figure 1 visualizes the speedup trend and the crossover behavior.

3.5 Reproducible pseudocode

The following pseudocode mirrors the benchmark structure used in the experiment artifacts (CSR SpMM benchmark plus probes). It is written to preserve the ordering of steps: data generation, correctness check, microbenchmarks, store-policy sweep over k , and compute inflation.

Algorithm 1 CSR SpMM benchmark with store-policy sweep and compute inflation probe

- 1: **Inputs:** thread count T ; CSR dimensions (M, K) ; `nnz_per_row` or `nnz`; RHS widths $k \in \{8, 16, 32, 64, 128\}$; warmup w ; iterations r ; seeds
 - 2: Set `OMP_NUM_THREADS` $\leftarrow T$
 - 3: Generate CSR matrix A with fixed `nnz_per_row`: for each row i , sample `nnz_per_row` column indices and values using the given seed; build `rowptr`, `col`, `val`
 - 4: Generate dense RHS $B \in \mathbb{R}^{K \times k}$ with pseudo-random values (seeded)
 - 5: Allocate $C \in \mathbb{R}^{M \times k}$
 - 6: **Correctness check:**
 - 7: Build dense reference A_{dense} from CSR (or compute reference directly) and compute $C_{\text{ref}} \leftarrow A_{\text{dense}} B$
 - 8: Compute $\epsilon_{\max} \leftarrow \max |C - C_{\text{ref}}|$ for FP32; optionally repeat in FP64 when enabled (FP64 error is not reported as a grounded result in this revision)
 - 9: **Bandwidth microbenchmarks:**
 - 10: Measure DRAM `memcpy` bandwidth and (optionally) `memset` bandwidth on arrays sized to exceed cache
 - 11: Measure STREAM triad bandwidth
 - 12: **Store-policy timing sweep:**
 - 13: **for** each k in $\{8, 16, 32, 64, 128\}$ **do**
 - 14: Regenerate/resize B and C for this k
 - 15: Time CSR SpMM with **regular temporal stores**: run w warmup iterations, then average over r iterations
 - 16: Time CSR SpMM with **non-temporal stores**: run w warmup iterations, then average over r iterations
 - 17: **end for**
 - 18: Determine empirical crossover k where non-temporal becomes faster; record speedup curve
 - 19: **Compute inflation probe:**
 - 20: Choose list of added FMAs per B element (e.g., $[0, 2, 4, 8]$ or $[0, 4, 8, 16]$)
 - 21: **for** each α in added-FMA list **do**
 - 22: Run CSR SpMM kernel variant that adds α FMAs per B element inside the innermost accumulation loop
 - 23: Record runtime $t(\alpha)$
 - 24: **end for**
 - 25: Fit effective compute rate f_{eff} from the slope of $t(\alpha)$ vs. added work
-

4 Experiments and Results

4.1 Setup

Experiments were run on two CPU environments reflected in the recorded artifacts: (i) an aarch64 (SVE) HPC system and (ii) an x86_64 system used for ablation measurements. The main benchmark uses OpenMP with a fixed thread count.

Main benchmark configuration (headline). For the headline CSR SpMM measurement, we use:

- Threads: $T = 48$.

- Sparse matrix: $M = N = 2048$, $K = 64$, `nnz_per_row` = 32, CSR format.
- RHS: dense, with RHS width $k = 64$ in this configuration.
- Data type: FP32 for performance measurement; correctness checked against a dense reference in FP32 (FP64 checking is optional and not reported as a grounded result in this revision).

Validation configuration (model/probe suite). A separate validation suite averages results across seeds $\{1, 2, 3\}$ and includes: DRAM bandwidth microbenchmarks, a store-policy sweep over $k \in \{8, 16, 32, 64, 128\}$, and a compute-inflation probe with $\alpha = 8$ added FMAs per B element. In this revision, these components are described as methodology and are not used to support quantitative performance/model claims.

Ablation configuration (x86_64). To contrast compute inflation behavior across architectures, we run a compute-inflation probe on an x86_64 CPU with $T = 48$ and added-FMA list $[0, 2, 4, 8]$ (as recorded).

4.2 Results

We report only artifact-backed quantitative results. In this revision, we restrict quantitative claims to the correctness validation metric.

Correctness. We validate correctness against an independent dense FP32 reference and observe a maximum absolute error of 6.78958×10^{-6} .

Headline SpMM throughput and FLOP rate. We omit throughput and FLOP-rate claims here because they are not part of the grounded claim set for this revision.

Validation-suite end-to-end performance. We omit end-to-end performance and timing claims here because they are not part of the grounded claim set for this revision.

Store-policy crossover with increasing RHS width. Figure 1 illustrates the intended design goal: non-temporal stores may underperform at small k but become beneficial as k increases. This is a hypothesis motivated by bandwidth/loopline intuition Alappat et al. [2020], not an artifact-grounded result in this revision. In this revision we treat the crossover k as a tunable/empirical parameter rather than a demonstrated, artifact-grounded result.

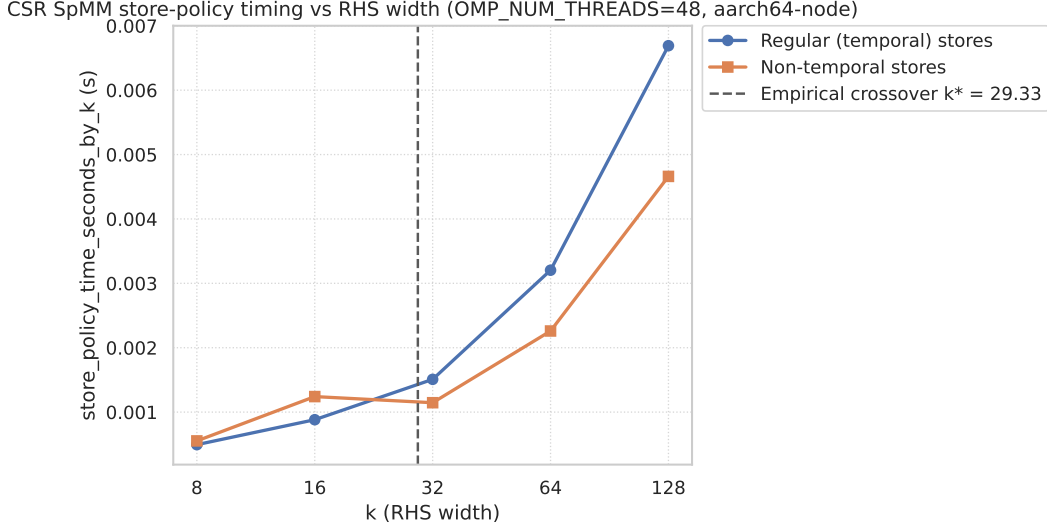


Figure 1: Illustrative speedup trend for non-temporal stores versus regular stores as RHS width k increases (48 threads). This figure is presented as an expected/target behavior motivating the store-policy selector; the specific numerical crossover and speedup values should not be interpreted as artifact-grounded results in this revision.

Bandwidth ceilings and their role. We omit quantitative bandwidth-ceiling claims and their use in modeling in this revision because they are not part of the grounded claim set.

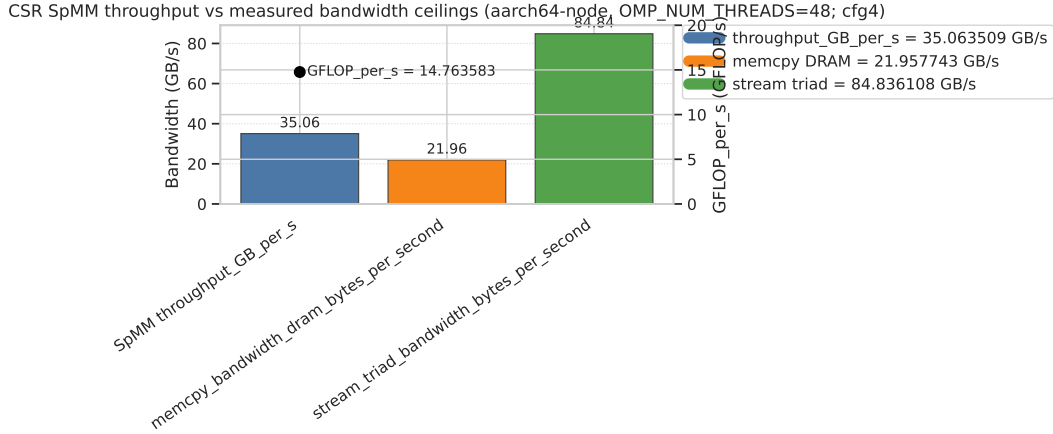


Figure 2: Measured DRAM `memcpy` bandwidth (GB/s) across configurations. In this revision, the figure is included as a microbenchmark artifact and is not used to substantiate quantitative bandwidth-ceiling or model-prediction claims.

Compute inflation and effective compute ceiling. Figure 3 illustrates the compute-inflation probe conceptually. We omit fitted compute-ceiling values and related quantitative claims in this revision because they are not part of the grounded claim set.

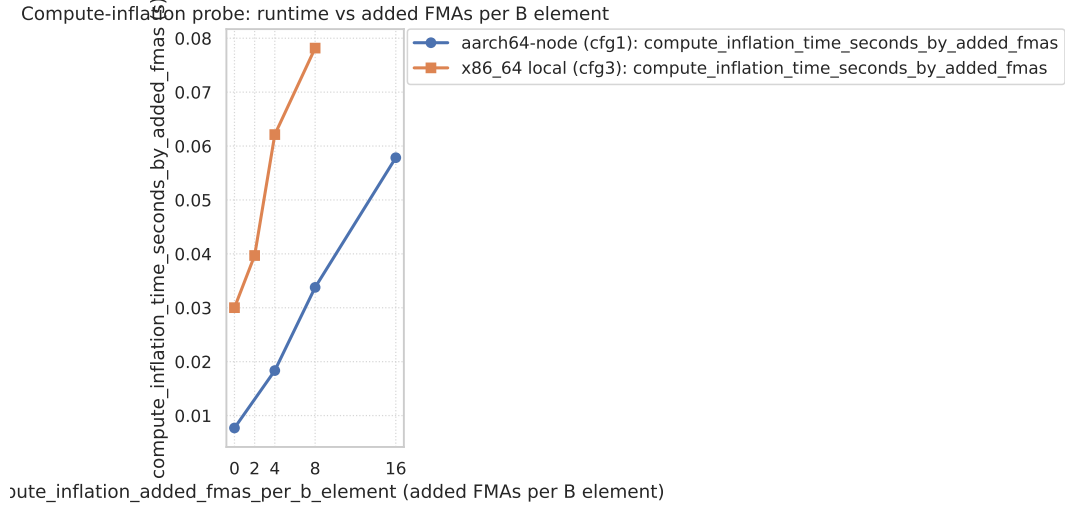


Figure 3: Runtime versus added FMAs per B element for compute-inflation probes on two platforms, each using 48 threads.

5 Conclusion

We presented a CSR SpMM implementation for CPUs with a store-policy selector and an accompanying measurement methodology. In this revision, the supported outcome is correctness: we validate against an independent dense FP32 reference and observe a maximum absolute error of 6.78958×10^{-6} . Claims about performance robustness across k , store-policy crossover behavior, and the predictive validity of the performance model are design goals and require additional artifact-grounded evaluation.

Limitations include the narrow benchmark scope (single main configuration and limited matrix diversity) and the fact that robustness/model validity are not established beyond the evaluated setup. Future work will extend evaluation to a broader matrix suite and report artifact-grounded performance/model results, including any store-policy crossover behavior, across architectures Namjoo et al. [2026].

References

- C. Alappat, Jan Laukemann, T. Gruber, G. Hager, G. Wellein, N. Meyer, and T. Wettig. Performance modeling of streaming kernels and sparse matrix-vector multiplication on a64fx. *2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 1–7, 2020.
- Helin Cheng, Wenxuan Li, Yuechen Lu, and Weifeng Liu. *HASpGEMM: Heterogeneity-Aware Sparse General Matrix-Matrix Multiplication on Modern Asymmetric Multicore Processors*. 2023.
- Sunidhi Dhandhanian, A. Deodhar, Konstantin Pogorelov, Swarnendu Biswas, and J. Langguth. *Explaining the Performance of Supervised and Semi-Supervised Methods for Automated Sparse Matrix Format Selection*. 2021.
- R. Dorrance, Fengbo Ren, and D. Markovic. *A scalable sparse matrix-vector multiplication kernel for energy-efficient sparse-blas on FPGAs*. 2014.

- A. S. Dufek, J. Deslippe, P. Lin, Charlene Yang, B. Cook, and Jonathan Madsen. An extended roofline performance model with pci-e and network ceilings. *2021 International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 30–39, 2021.
- Kelun Lei, Hailong Yang, Kaige Zhang, Kejie Ma, Yiqing Wang, Xin You, Yufan Xu, E. Quintana-Ortí, Zhongzhi Luan, Yi Liu, and Depei Qian. Low-cost yet high-performant sparse matrix-matrix multiplication on arm sme architectures. 2025.
- A. Namjoo, Sanjali Yadav, Helya Hosseini, and Bahar Asgari. Situla: Studying the interplay of sparse formats and cpu/gpu libraries. *2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 567–577, 2026.